

Grundlagen der Informatik

Boole'sche Algebra

Ferdinand Pöll, Tobias Walter

Hochschule Mainz University of Applied Sciences

ferdinand.poell@lba.hs-mainz.de, tobias.walter@hs-mainz.de

13. Oktober 2025

- 1 Aussagenlogik
- 2 Boolesche Algebra
- 3 Axiome
- 4 De Morgansche Gesetze
- 5 Boolesche Schaltung
- 6 Addierwerk
- 7 Arithmetic Logic Unit

- Die Aussagenlogik ist ein Teilgebiet der Logik, das sich mit Aussagen und deren Verknüpfung durch Junktoren befasst, ausgehend von strukturlosen Elementaraussagen (Atomen), denen ein Wahrheitswert zugeordnet wird.
- In der klassischen Aussagenlogik wird jeder Aussage genau einer der zwei Wahrheitswerte „wahr“ und „falsch“ zugeordnet.
- Der Wahrheitswert einer zusammengesetzten Aussage lässt sich ohne zusätzliche Informationen aus den Wahrheitswerten ihrer Teilaussagen bestimmen.



- George Boole war ein englischer Mathematiklehrer, der Mitte des 19. Jahrhunderts sich mit den formalen Strukturen binärer Ausdrücke beschäftigte.
- Er entwickelte die heute nach ihm benannte Boole'sche Algebra.
- In der boolschen Algebra existieren lediglich die zwei Werte 0 (falsch) und 1 (wahr)

- Zwei Variablen $a, b \in \mathbb{B}$
- Elementaraussagen: keine aussagenlogischen Verknüpfungen
- Drei Operationen:
 - Definition von 2-stelligen Operationen: AND, OR
 - Definition von 1-stelligen Operationen: NOT

- Zwei Variablen $a, b \in \mathbb{B}$
- Elementaraussagen: keine aussagenlogischen Verknüpfungen
- Drei Operationen:
 - Definition von 2-stelligen Operationen: AND, OR
 - Definition von 1-stelligen Operationen: NOT

Beispiele für Elementaraussagen:

- A_1 : München ist 781 km von Hamburg entfernt.
- A_2 : 9 ist durch 3 teilbar.
- A_3 : Eintracht Frankfurt wird in der nächsten Saison deutscher Fußballmeister.
- A_4 : Alle Autos sind grün.

- Eine Disjunktion ist eine zusammengesetzte Aussage, die behauptet, dass mindestens eine ihrer Teilaussagen wahr ist.
- Wird auch als nichtausschließendes Oder bezeichnet.
- Sprechweise: „A oder B“; genauer: „A oder B oder beide“, häufig in juristischen oder medizinischen Texten: „A und/oder B“
- $A \vee B := \max(x, y)$
- Schreibweise: $A \vee B$ oder $A + B$
- Auf Englisch und in der Schaltalgebra auch A OR B

$\vee$	0	1
0	0	1
1	1	1

- Eine Konjunktion ist eine aus zwei Aussagen zusammengesetzte Aussage, die die Wahrheit all ihrer Teilaussagen behauptet.
- $A \wedge B := \min(x, y)$
- Sprechweise: A und B
- Schreibweise: $A \wedge B$ oder $A * B$
- Auf Englisch und in der Schaltalgebra auch A AND B

$\wedge$	0	1
0	0	0
1	0	1

- Die Verneinung bzw. Negation (auch: Satzverneinung, äußere Verneinung, kontradiktorisches Gegenteil) einer Aussage A ist diejenige Aussage $\neg A$, die genau dann wahr ist, wenn A falsch ist, und die genau dann falsch ist, wenn A wahr ist.
- Sprechweise: Nicht A
- Schreibweise: $\neg A$ oder $\overline{A}$

a	$\neg a$
0	1
1	0

- C_1 : 9 ist durch 3 teilbar und 9 ist eine Quadratzahl.
- C_2 : 9 ist nicht durch 3 teilbar und 9 ist eine Quadratzahl.
- C_3 : 9 ist durch 3 teilbar und 9 ist keine Quadratzahl.
- C_4 : 9 ist nicht durch 3 teilbar und 9 ist keine Quadratzahl

- C_1 : 9 ist durch 3 teilbar und 9 ist eine Quadratzahl.
- C_2 : 9 ist nicht durch 3 teilbar und 9 ist eine Quadratzahl.
- C_3 : 9 ist durch 3 teilbar und 9 ist keine Quadratzahl.
- C_4 : 9 ist nicht durch 3 teilbar und 9 ist keine Quadratzahl

A	B	$A \wedge B$
wahr	wahr	wahr
falsch	wahr	falsch
wahr	falsch	falsch
falsch	falsch	falsch

- Verschmelzungsgesetz: $(x \vee y) \wedge x = x$
- Wertetabelle als Beweis der Aussage

x	y	$x \vee y$	$(x \vee y) \wedge x$	x
0	0	0	0	0
0	1	1	0	0
1	0	1	1	1
1	1	1	1	1

Axiome der booleschen Algebra

Tabelle 4.1

Rechenregeln in der booleschen Algebra

Name	Formel	Kurzschreibweise
Kommutativgesetze	$a * b = b * a$ $a + b = b + a$	$ab = ba$
Assoziativgesetze	$a * (b * c) = (a * b) * c$ $a + (b + c) = (a + b) + c$	$a(bc) = (ab)c$
Distributivgesetze	$a * (b + c) = (a * b) + (a * c)$ $a + (b * c) = (a + b) * (a + c)$	$a(b + c) = ab + ac$ $a + bc = (a + b)(a + c)$
Identitätsgesetze	$a * 1 = a$ $a + 0 = a$	$a1 = a$
Null-/Einsgesetze	$a * 0 = 0$ $a + 1 = 1$	$a0 = 0$
Komplementärgesetze	$a * \bar{a} = 0$ $a + \bar{a} = 1$	$a\bar{a} = 0$ $a + \bar{a} = 1$
Idempotenzgesetze	$a * a = a$ $a + a = a$	$aa = a$
Verschmelzungsgesetze	$a * (a + b) = a$ $a + (a * b) = a$	$a(a + b) = a$ $a + ab = a$
De Morgan'sche Gesetze	$\overline{(a * b)} = \bar{a} + \bar{b}$ $\overline{(a + b)} = \bar{a} * \bar{b}$	$\overline{ab} = \bar{a} + \bar{b}$ $\overline{a + b} = \bar{a}\bar{b}$
Doppeltes Negationsgesetz	$\overline{(\bar{a})} = a$	$\bar{\bar{a}} = a$

- Das ausschließende Oder (Kontravalenz oder Antivalenz) besagt, dass genau eine der beiden von ihm verknüpften Aussagen wahr ist.
- Sprechweise: Entweder A oder B
- Schreibweise: $\neg(A \leftrightarrow B)$ oder $A \text{ XOR } B$ oder $A \oplus B$

XOR	0	1
0	0	1
1	1	0

- Das ausschließende Oder (Kontravalenz oder Antivalenz) besagt, dass genau eine der beiden von ihm verknüpften Aussagen wahr ist.
- Sprechweise: Entweder A oder B
- Schreibweise: $\neg(A \leftrightarrow B)$ oder $A \text{ XOR } B$ oder $A \oplus B$
- Konstruiert sich wie folgt: $(x \wedge \neg y) \vee (\neg x \wedge y)$

XOR	0	1
0	0	1
1	1	0

Verneinung der Konjunktion „A und B“

- Beispiel: Es ist nicht der Fall, dass A und B zutreffen
- $\neg(A \wedge B) \equiv \overline{(A \wedge B)}$
- $(\neg A \vee \neg B)$

Verneinung der Disjunktion „A oder B (oder beides)“

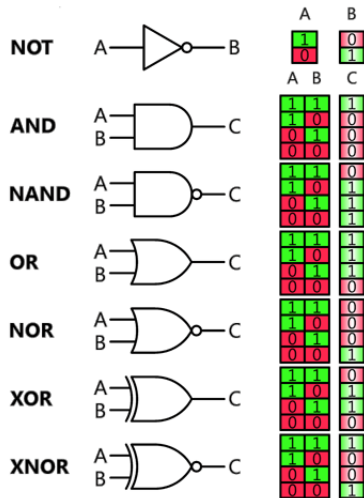
- Beispiel: Es ist nicht der Fall, dass A oder B zutrifft
- $\neg(A \vee B) \equiv \overline{(A \vee B)}$
- $\neg A \wedge \neg B$

Zeigen Sie, dass folgende Gleichung gilt:

$$(a + b) \cdot (b + c) \cdot (c + a) = (a \cdot b) + (b \cdot c) + (c \cdot a)$$

- ① durch Anwendung einer Wertetabelle
- ② durch Anwendung von Booleschen Gesetzen

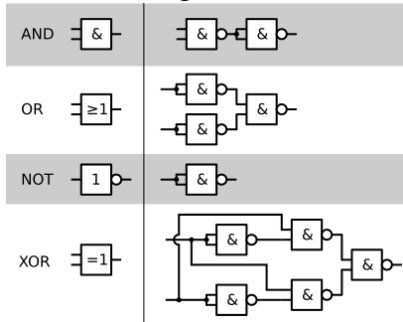
Umsetzung in Schaltungen



Umsetzung in Schaltungen - Einheitliche Elemente

	AND	OR	NOT	XOR	NAND	NOR	XNOR
IEC 60617-12							
US ANSI 91-1984							
DIN 40700							

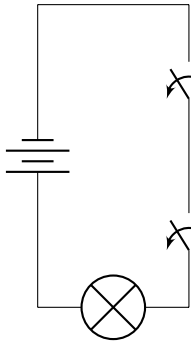
Bei der Erstellung integrierter Schaltkreise ist es nötig einheitliche Elemente zu verwenden, so dass stets die gleiche Maske in der Entwicklung verwendet werden kann.



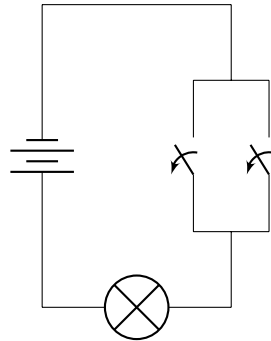
Es lassen sich folgende Sätze aufstellen:

- Satz: Alle zweistelligen booleschen Funktionen können mit Hilfe der Negation ($-$), der Konjunktion ($*$) und der Disjunktion ($+$) dargestellt werden.
- Satz: Alle zweistelligen booleschen Funktionen können entweder mit Hilfe der Negation und der Konjunktion, oder mit Hilfe der Negation und der Disjunktion dargestellt werden.
- Satz: Alle zweistelligen booleschen Funktionen können entweder mit Hilfe der NAND-Verknüpfung oder mit Hilfe der NOR-Verknüpfung dargestellt werden.

Serienschaltung

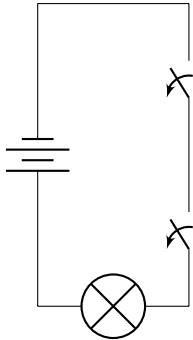


Parallelschaltung

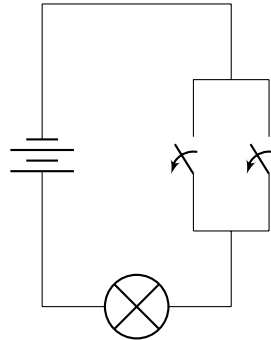


Boolesche Schaltung

Serienschaltung (AND Operation)

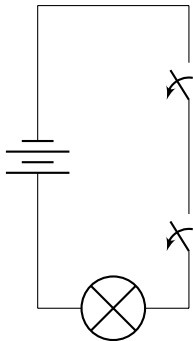


Parallelschaltung

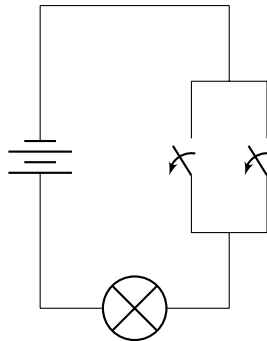


Boolesche Schaltung

Serienschaltung (AND Operation)



Parallelschaltung (OR Operation)



—— Input ——→ **Deterministischer Rechner** —— Output ——→

Seien $n, m \in \mathbb{N}$, $m \geq 1$. Dann heißt die Funktion $F : B^n \rightarrow B^m$ Schaltfunktion.
Beispiele von Schaltfunktionen:

- Addition von zwei 16-stelligen Dualzahlen

$$A : B^{32} \rightarrow B^{17}$$

- Multiplikation von zwei 16-stelligen Dualzahlen

$$A : B^{32} \rightarrow B^{32}$$

- Sortieren von 30 16-stelligen Dualzahlen

$$A : B^{480} \rightarrow B^{480}$$

- Primzahltest für eine 480-stellige Dualzahl

$$A : B^{480} \rightarrow B$$

- Eine Schaltfunktion $F : B^n \rightarrow B$ heißt (n -stellige) Boolesche Funktion (teilweise auch allgemeiner $F : B^n \rightarrow B^m$)
- Jede Schaltfunktion ist durch eine Folge von Booleschen Funktionen beschreibbar:
 - $F : B^n \rightarrow B^m, F(x_1, \dots, x_n) = (y_1, \dots, y_m)$
 - $f_i : B^n \rightarrow B, f_i(x_1, \dots, x_n) = y_i \forall i \in \{1, \dots, m\}$
 - $F(x_1, \dots, x_n) = (f_1(x_1, \dots, x_n), \dots, f_m(x_1, \dots, x_n))$
- Frage: Wie viele n -stellige Boolesche Funktionen gibt es?

1-stellige Boolesche Funktionen

x	$f_0(x)$	$f_1(x)$	$f_2(x)$	$f_3(x)$
0	0	0	1	1
1	0	1	0	1

$$f_0(x) \equiv 0, f_1(x) = x, f_2(x) \neg x, f_3(x) \equiv 1$$

2-stellige Boolesche Funktionen

x	y	$f_0(x)$ $x \cdot \neg x$	$f_1(x)$ $x \cdot y$	$f_2(x)$ $x \cdot \neg y$	$f_3(x)$ x	$f_4(x)$ $\neg x \cdot y$	$f_5(x)$ y	$f_6(x)$ $x \oplus y$	$f_7(x)$ $x + y$
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

x	y	$f_8(x)$ $\neg(x + y)$	$f_9(x)$ $x \leftrightarrow y$	$f_{10}(x)$ $\neg y$	$f_{11}(x)$ $x + \neg y$	$f_{12}(x)$ $\neg x$	$f_{13}(x)$ $\neg x + y$	$f_{14}(x)$ $\neg(x \cdot y)$	$f_{15}(x)$ $x + \neg x$
0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

$f_1(x)$: Konjunktion (AND), $f_6(x)$: Antivalenz (XOR), $f_7(x)$: Disjunktion(OR), $f_8(x)$:
 Peircescher Pfeil (NOR), $f_9(x)$: Äquivalenz, $f_{13}(x)$: Implikation, $f_{14}(x)$: Shefferscher Strich
 (NAND)

- Anzahl n-stelliger Boolesche Funktionen
 - $n=1$: $2^2 = 4$
 - $n=2$: $2^4 = 16$
 - $n=3$: $2^8 = 256$
 - $n=4$: $2^{16} = 65.536$
- Für jedes $n \in \mathbb{N}$ mit $n \geq 1$ gibt es 2^n n-stellige Boolesche Funktionen
- Frage: Wie kann man eine beliebige n-stellige Booleschen Funktion aus wenigen Grundfunktionen darstellen?

Einschlägiger Index

i	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

- Einschlägige Indizes zu f : sind 3, 5, 7
- Jede n -stellige Boolesche Funktion lässt sich durch eine Funktionstafel mit 2^n Zeilen darstellen, wobei in Zeile i $0 \leq i \leq 2^n - 1$ die Dualdarstellung von i steht, i heißt ein Index zu f
- i heißt einschlägiger Index, falls $f(i_1, \dots, i_n) = 1$

i	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

- Minterm zu $f : m_3(x_1, x_2, x_3) = \neg x_1 \cdot x_2 \cdot x_3$
- Minterm zu $f : m_6(x_1, x_2, x_3) = x_1 \cdot x_2 \cdot \neg x_3$
- Sei $i = (i_1, \dots, i_n)_2$ ein Index von $f : B^n \rightarrow B$
- Die Funktion $m_i(x_1, \dots, x_n) = x_1^{i_1} \cdot x_2^{i_2} \cdot \dots \cdot x_n^{i_n}$ heißt i-ter Minterm von f , Kurzschreibweise: m_i
- Beobachtung: Minterm m_i ist genau dann 1, wenn das Argument $(x_1, \dots, x_n)$ die Dualdarstellung von i liefert
 - $m_3 \Leftrightarrow x_1 = 0 \wedge x_2 = 1 \wedge x_3 = 1$
 - $m_6 \Leftrightarrow x_1 = 1 \wedge x_2 = 1 \wedge x_3 = 0$

Disjunktive Normalform (DNF)

i	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

- $f(x_1, x_2, x_3) = m_1 + m_2 + m_3 = \neg x_1 \cdot x_2 \cdot x_3 + x_1 \cdot \neg x_2 \cdot x_3 + x_1 \cdot x_2 \cdot x_3$
- Darstellungssatz für Boolesche Funktionen: Jede Boolesche Funktion $f : B^n \rightarrow B$ ist eindeutig darstellbar als Summe der Minterme ihrer einschlägigen Indizes

i	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

- Maxterm zu
 $f : M_3(x_1, x_2, x_3) = \neg m_3 = \neg(\neg x_1 \cdot x_2 \cdot x_3) = x_1 + \neg x_2 + \neg x_3$
- Maxterm zu
 $f : M_6(x_1, x_2, x_3) = \neg m_6 = \neg(x_1 \cdot x_2 \cdot \neg x_3) = \neg x_1 + \neg x_2 + x_3$
- Sei $i = (i_1, \dots, i_n)_2$ ein Index von $f : B^n \rightarrow B$
- Die Funktion $M_i(x_1, \dots, x_n)$ heißt i-ter Maxterm von f, Kurzschreibweise: M_i
- Beobachtung: Maxterm M_i ist genau dann 0, wenn das Argument $(x_1, \dots, x_n)$ die Dualdarstellung von i liefert:
 $M_3 = 0 \Leftrightarrow x_1 = 0 \wedge x_2 = 1 \wedge x_3 = 1$

Konjunktive Normalform (KNF)

i	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

- Jede Boolesche Funktion $f : B^n \rightarrow B$ ist eindeutig darstellbar als Produkt der Maxterme ihrer nicht einschlägigen Indizes
- Aufgabe: Bestimmen Sie die KNF dieser Booleschen Funktion

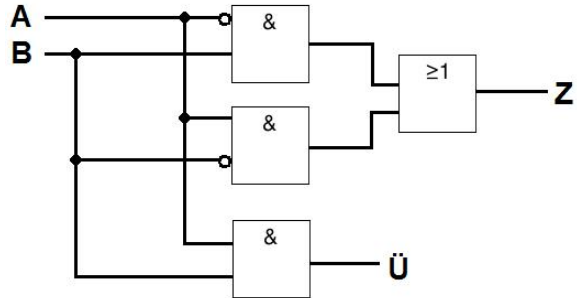
Halbaddierer

- Ein Halbaddierer (engl. „half adder“) ist ein Schaltnetz.
- Er besteht aus zwei Eingängen und zwei Ausgängen.
- Mit einem Halbaddierer kann man zwei einstellige Binärzahlen addieren.
- Dabei liefert der Ausgang s (engl. sum – „Summe“) die rechte und der Ausgang c (engl. carry – „Übertrag“) die linke Stelle des Ergebnisses.

x	y	Übertrag c	Summe s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$c = x \wedge y$$

$$s = x \vee y = x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge y)$$



Volladdierer

- Ein Volladdierer (englisch full adder) ist ein Schaltnetz.
- Es besteht aus drei Eingängen (x , y und c_{in}) und zwei Ausgängen (s und c_{out}).
- Mit einem Volladdierer kann man drei einstellige Binärzahlen addieren. Dabei liefert der Ausgang s (engl. sum – Summe) die niederwertige Stelle des Ergebnisses, der Ausgang c_{out} (engl. carry (output) – Übertrag (Ausgang)) die höherwertige.

x	y	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

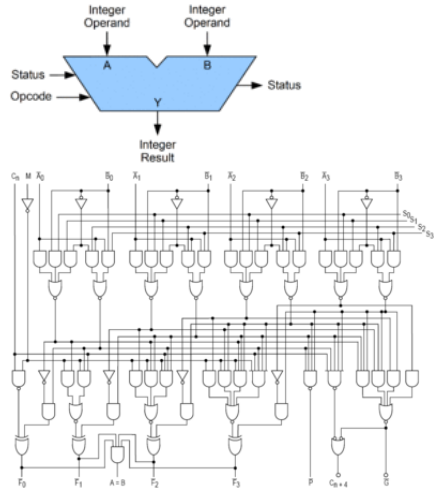
$$c_{out} = (\bar{x} \wedge y \wedge c_{in}) \vee (x \wedge \bar{y} \wedge c_{in}) \vee (x \wedge y \wedge \bar{c}_{in}) \vee (x \wedge y \wedge c_{in}) = (c_{in} \wedge (x \oplus y)) \vee (x \wedge y)$$
$$s = x \oplus y \oplus c_{in}$$

Arithmetic Logic Unit

Eine ALU kann zwei Binärwerte mit gleicher Stellenzahl (n) miteinander verknüpfen. Man spricht von n -Bit-ALUs.

Die ALU berechnet arithmetische und logische Funktionen. Sinnvollerweise kann sie mindestens folgende Minimaloperationen durchführen:

- Arithmetisch:
 - Addition (ADD)
- Logisch:
 - Negation (NOT)
 - Konjunktion (Und-Verknüpfung, AND)



Meist werden aber weitaus mehr Befehle unterstützt:

- Arithmetisch:
 - Subtraktion (SUB)
 - Vergleich (compare, CMP)
 - Multiplikationen (MUL) mittels Multiplizierer
 - Division
 - Decimal adjust after addition (Dezimal-Angleichung nach Addition)
- Logisch:
 - Disjunktion (Oder-Verknüpfung, OR)
 - Kontravalenz (Exklusiv-Oder-Verknüpfung, XOR, EOR)
 - Rechts- und Linksverschiebung (Rechts-, Linksshift, ASR – arithmetische Shift rechts, ASL – arithmetische Shift links, LSR – logisches Verschieben nach rechts, LSL – logisches Verschieben nach links)
 - Links- und Rechtsrotation (ROL, ROR)
 - Register-Manipulationen und Bit-Veränderungen (Bits setzen, löschen und testen)
 - Umsortieren von Bits und Bytes, AES-Befehle, CRC-Befehle

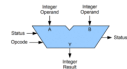
Arithmetic Logic Unit

A / B Eingehende Integer Werte

Result Ergebnis der Operation, die im Opcode definiert ist.

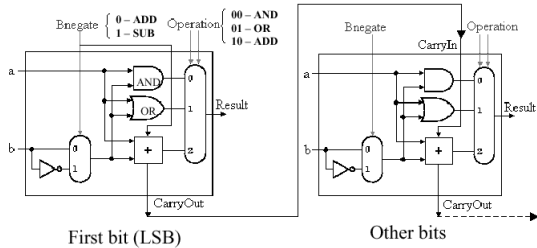
Status In/Out Dieses Register enthält meistens vier Statusbits. Die einzelnen Werte des Statusregisters können von Maschinensprachebefehlen abgefragt werden und damit den weiteren Programmverlauf beeinflussen (z. B. bedingte Sprünge).

Opcode Definition der Operation. Schaltet in der ALU die entsprechenden Operationen und wird durch den Befehl im Program definiert.



- Übertragsbit (C für engl. carry bit): Zeigt den Übertrag an, der bei Addition der n-ten Bits (der höchsten Stelle) der Operanden entsteht. Es wird als Kennzeichen für Bereichsüberschreitung bei Betrachtung des Ergebnisses als vorzeichenlose Zahl (nur positive Ergebnisse) verwendet. Wird in 2er-Komplement-Darstellung gerechnet (auch negative Zahlen möglich), so spielt das Carry-Bit für die Bereichsüberschreitung keine direkte Rolle.
- Überlaufbit (V für engl. overflow bit): Zeigt Zahlenbereichsüberschreitung bei 2er-Komplement-Rechnung an (Übertrag der zweithöchsten Stelle der Operanden). Weitere mögliche Bezeichnungen sind neben V noch OF oder O.
- Nullbit (Z für engl. zero bit): Zeigt an, ob das Ergebnis der vorhergehenden Rechenoperation Null ist (wird 1, wenn Inhalt des Akkumulatorregisters 0 ist).
- Negativbit (N): Wird gesetzt, wenn das Ergebnis der Operation als negative Zahl zu interpretieren sein kann (oberstes Bit des Akkumulatorregisters = 1).

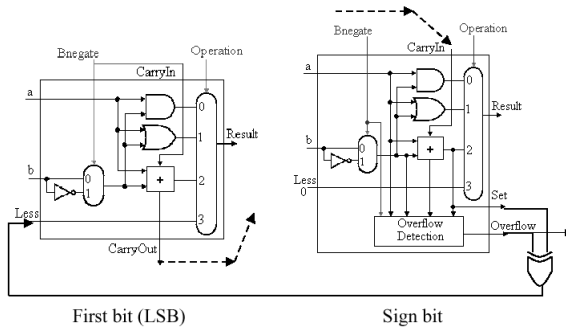
Beispiel: MIPS ALU Design



Operations: AND, OR, ADD, SUB

Control lines: 000 001 010 110

MIPS design with SLT



- Wir haben bisher den Ripple-Carry-Addierer besprochen, der das Carry-Bit von Stufe i zu Stufe $i + 1$ ausbreitet.
- Für eine N -Bit-Eingabe eine $O(N)$ -Zeit erforderlich ist, um den Übertrag auf das höchstwertige Bit zu übertragen. Im Gegensatz dazu verwendet der schnellste N -Bit-Addierer $O(\log_2 N)$ in einer baumstrukturierten Konfiguration mit $N-1$ -Ein-Bit-Addierern.

Carry Look Ahead ALU

